Chapter4 Relational Algebra

CHAPTER OVERVIEW

Understanding relational algebra is essential for grasping how query optimizers execute, transform, and evaluate database queries under the hood. The chapter covers the fundamental definitions, five primary vocabulary terms, and the core operations:

- **Unary Operations:** Selection (σ), Projection (π), and Rename (ρ).
- **Set-Theoretic Operations:** Union ($\cup$), Intersection ($\cap$), Set Difference ($-$), and Cartesian Product ($\times$).
- **Joins & Advanced Concepts:** How foreign keys and conditions transform raw Cartesian products into meaningful joins across normalized relations.

🎯 **Learning Outcomes**

At the end of this chapter, you will be able to:

1. Explain the difference between declarative SQL statements and procedural relational algebra expressions.
2. Define and correctly apply core terms: *Relation*, *Tuple*, *Attribute*, *Degree* (number of columns), and *Cardinality* (number of rows).
3. Filter relation rows using the **Selection (σ)** operator with logical and comparison conditions.
4. Filter relation columns using the **Projection (π)** operator and understand its automatic duplicate elimination property.
5. Rename relations and attributes using the **Rename (ρ)** operator to resolve column name conflicts during complex operations.
6. Evaluate **Union ($\cup$)**, **Intersection ($\cap$)**, and **Set Difference ($-$)** while enforcing union-compatibility rules (matching degrees and compatible attribute domains).
7. Compute the **Cartesian Product ($\times$)** of two relations and calculate the resulting degree and cardinality.
8. Explain why unconditioned Cartesian products create invalid data pairs and how primary/foreign key relationships filter these into meaningful join operations.
9. Convert standard SQL queries (`SELECT` , `WHERE` , `JOIN` , `UNION` , `EXCEPT` , `INTERSECT`) into equivalent relational algebra expressions.

1 What Is Relational Algebra?

SQL is a *declarative* language: you say *what* you want; the DBMS decides *how* to compute it. The "how" is determined by translating your SQL into **relational algebra** — a *procedural* formal language where every operation is an explicit step. Understanding relational algebra means understanding how the query optimizer thinks, why certain queries are fast, and what is happening beneath every `JOIN` you write.

📖 DEFINITION 1: **Relational Algebra**

Relational algebra is a **procedural formal query language** for relations. Each *operation* takes one or two relations as input and returns a new relation as output.

ⓘ **Closure property**

The result of every relational algebra operation is itself a relation. This means operations can be freely composed — the output of one becomes the input of the next.

$$Expression = Opk(\ldots Op2(Op1(R1, R2))\ldots)$$

There are **6 fundamental operations** (from which all others can be derived) and several convenient **additional (derived) operations**.

Group	Sub-Category / Type	Operations
Fundamental (Basic)	Unary	Selection, Projection, Rename
	Set-Theoretic	Union, Set Difference, Cartesian Product
Derived (Additional)	Set-Theoretic	Set Intersection
	Joins	Natural Join, Theta Join, Outer Joins, Semijoins
	Division	Division
Extended	Aggregations	Aggregate Functions (Group By)

2 Fundamental Concepts in Relational Algebra

Before you can work with relational algebra operators, you need to know the vocabulary. The good news is that there are only five terms to remember.

- Relation is just a table. It has rows and columns, and it represents a set of data about a specific entity - like employees or departments. In relational algebra, every operation takes one or more relations as input and produces a relation as output.
- Tuple is a single row in that table. If your `Employees` table has 10 rows, it has 10 tuples. Each tuple represents one record - one employee, in this case.
- Attribute is a column. The `Employees` table might have attributes like `employee_id`, `name`, and `department_id`. Attributes define what kind of data each tuple holds.
- Degree is the number of attributes in a relation. If `Employees` has five columns, its degree is 5. This matters when you're combining relations - some operations require matching degrees.
- Cardinality is the number of tuples in a relation. If your `Employees` table has 200 rows, its cardinality is 200. Cardinality changes as you add or remove rows; degree changes only when you add or remove columns.

3 Fundamental Operations: Unary Operations (Operate on a single relation)

3.1 The SELECT Operation — σ (Filter Rows)

The **SELECT** operation is used to select a subset of the tuples from a relation that satisfy a **selection condition**. One can consider the SELECT operation to be a filter that keeps only those tuples that satisfy a qualifying condition. The SELECT operation can also be visualized as a horizontal partition of the relation into two sets of tuples-those tuples that satisfy the condition and are selected, and those tuples that do not satisfy the condition and are discarded.

$$\sigma_{\text{condition}}(\text{R})$$

RELATIONAL ALGEBRA (RELAX)

```
σ condition (R)
```

SQL EQUIVALENT

```
SELECT * FROM R WHERE condition
```

Property	Value
Input	Relation R
Output columns	Same as R (degree unchanged)
Output rows	All tuples where condition is true (cardinality ≤ |R|)
Condition operators	=,≠,<,>,≤,≥,combined with ∧ (AND), ∨ (OR), ¬ (NOT)

📝 EXAMPLE 1: Selection by Salary Threshold

List all employees whose salary is greater than 65,000.

✅ Show Solution

$$\sigma_{\text{Salary}>65000}(\text{Employee})$$

RELATIONAL ALGEBRA (RELAX)

```
σ Salary > 65000 (Employee)
```

SQL EQUIVALENT

```
SELECT * FROM Employee WHERE Salary > 65000
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Salary > 65000 (Employee)
```

EXAMPLE 2: Selection by Department ID

List all employees who belong to department 1.

Show Solution

$$\sigma_{Dno=1} Employee$$

RELATIONAL ALGEBRA (RELAX)

```
σ Dno = 1 (Employee)
```

SQL EQUIVALENT

```
SELECT * FROM Employee WHERE Dno = 1
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Dno = 1 (Employee)
```

EXAMPLE 3: Selection with Logical NOT

List all employees who do not work in department 2.

Show Solution

$$\sigma_{\neg(Dno=2)} Employee$$

RELATIONAL ALGEBRA (RELAX)

```
σ ¬ Dno = 2 (Employee)
```

SQL EQUIVALENT

```
SELECT * FROM Employee WHERE NOT Dno = 2
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ ¬ Dno = 2 (Employee)
```

EXAMPLE 4: Compound Selection with Logical AND

List all employees who work in department 1 AND earn more than 60,000.

Show Solution

$$\sigma_{\text{Dno}=1 \wedge \text{Salary}>60000}(\text{Employee})$$

RELATIONAL ALGEBRA (RELAX)

```
σ Dno = 1 ∧ Salary > 60000 (Employee)
```

SQL EQUIVALENT

```
SELECT * FROM Employee WHERE Dno = 1 AND Salary > 60000
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Dno = 1 ∧ Salary > 60000 (Employee)
```

EXAMPLE 5: Pattern Matching (Starts With)

Select all employees whose first name starts with the letter 'H'.

Show Solution

$$\sigma_{Fname\ LIKE\ 'H\%'}(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
σ Fname LIKE 'H%' (Employee)
```

SQL EQUIVALENT

```
SELECT * FROM Employee WHERE Fname LIKE 'H%'
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Fname LIKE 'H%' (Employee)
```

📝 EXAMPLE 6: Pattern Matching (Contains)

Select all employees whose first name contains the letter 'm'.

✅ Show Solution

$$\sigma_{Fname\ LIKE\ '\%m\%'}(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
σ Fname LIKE '%m%' (Employee)
```

SQL EQUIVALENT

```
SELECT * FROM Employee WHERE Fname LIKE '%m%'
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Fname LIKE '%m%' (Employee)
```

3.2 The PROJECT Operation— π (Filter Columns)

While Selection operation works on rows, similarly projection operation of relational algebra works on columns. It basically allows us to pick specific columns from a given relational table based on the given condition and ignoring all the other remaining columns.

> **Note**
>
> By Default, projection operation removes **duplicate values**.

$$\pi_{A_1, A_2, \ldots, A_k}(R)$$

RELATIONAL ALGEBRA (RELAX)

```
π A_1, A_2, A_k (R)
```

SQL EQUIVALENT

```
SELECT DISTINCT A_1, A_2, A_k FROM R
```

EXAMPLE 7: Simple Projection

List the first name, last name, and salary of every employee.

Show Solution

$$\pi_{\text{Fname}, \text{Lname}, \text{Salary}}(\text{Employee})$$

RELATIONAL ALGEBRA (RELAX)

```
π Fname, Lname, Salary (Employee)
```

SQL EQUIVALENT

```
SELECT DISTINCT Fname, Lname, Salary FROM Employee
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π Fname, Lname, Salary (Employee)
```

EXAMPLE 8: Combined Select + Project

List the first name and salary of all female employees earning above 60,000.

Show Solution

$$\pi_{\text{Fname,Salary}}(\sigma_{\text{Gender}='F' \wedge \text{Salary}>60000}(\text{Employee}))$$

RELATIONAL ALGEBRA (RELAX)

```
π Fname, Salary (σ Gender = 'F' ∧ Salary > 60000 (Employee))
```

SQL EQUIVALENT

```
SELECT DISTINCT Fname, Salary FROM (SELECT * FROM Employee WHERE
Gender = 'F' AND Salary > 60000) AS t1
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π Fname, Salary (σ Gender = 'F' ∧ Salary > 60000 (Employee))
```

EXAMPLE 9: Combined Select + Project

Retrieve Fname as FIRST_NAME of all employees

Show Solution

$$\pi_{FIRST_NAME \leftarrow Fname}(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
π FIRST_NAME (ρ FIRST_NAME←Fname (Employee))
```

SQL EQUIVALENT

```
SELECT DISTINCT FIRST_NAME FROM (SELECT * FROM (SELECT Fname AS
FIRST_NAME FROM Employee) AS src) AS t1
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π FIRST_NAME (ρ FIRST_NAME←Fname (Employee))
```

3.3 The Rename Operation— ρ (Renames a relation or its attribute names)

Rename operator basically allows you to give a temporary name to a specific relational table or to its columns. It is very useful when we want to avoid ambiguity, especially in complex Queries. Rename is a unary operation used for renaming attributes of a relation.

Rename Relation

$$\rho_S(R)$$

RELATIONAL ALGEBRA (RELAX)

```
ρ S (R)
```

SQL EQUIVALENT

```
SELECT * FROM R AS S
```

Rename Attributes

$$\rho_{att1_new \leftarrow att1, att2_new \leftarrow att2}(R)$$

RELATIONAL ALGEBRA (RELAX)

```
ρ att1_new←att1, att2_new←att2 (R)
```

Rename Relation and Attributes

$$\rho_{R_new}(\rho_{att1_new \leftarrow att1}(\pi_{att1,att2}(R)))$$

RELATIONAL ALGEBRA (RELAX)

```
ρ R_new (ρ att1_new←att1 (π att1, att2 (R)))
```

📝 EXAMPLE 10:

Create a named subexpression `Supervisor` containing SSN, first name, and last name of all employees, using shorter column names.

✅ Show Solution

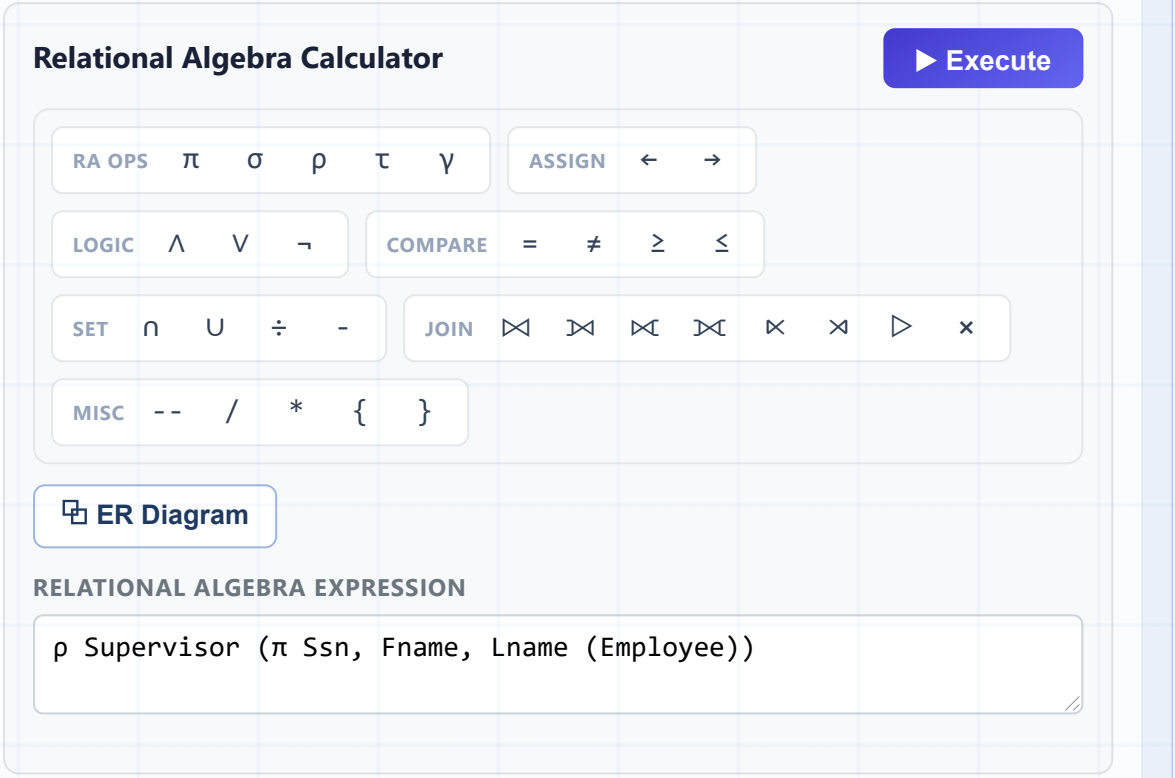
$$\rho_{Supervisor}(\pi_{Ssn,Fname,Lname}(Employee))$$

RELATIONAL ALGEBRA (RELAX)

```
ρ Supervisor (π Ssn, Fname, Lname (Employee))
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT DISTINCT Ssn, Fname, Lname FROM Employee) AS
Supervisor
```



4 Set-Theoretic Operations (Operate on two relations)

Set-theoretic operations in relational algebra are binary operators that combine or compare tuples from two relations, treating each relation as a formal set of tuples.

let's $R(A_1, A_2, ..., A_n)$and $S(B_1, B_2, ..., B_n)$are relations. The Set Compatibility Rules - ∪,∩, and − all require union-compatible operands:

1. Degree(R) = Degree(S): have the same number of attributes.
2. Domain(A_i) = Domain(B_i) , for 1<= i <=n: Corresponding attributes must have **compatible domains**(same or convertible types).

4.1 Union — ∪ (Combine Two Relations)

📖 DEFINITION 2: **Union (R ∪ S)**

Returns all tuples that belong to relation R, relation S, or both. The **result takes the attribute names from the left operand**(R)

$$R \cup S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ∪ S)
```

SQL EQUIVALENT

```
SELECT * FROM R
UNION
SELECT * FROM S
```

📝 EXAMPLE 11: — Union

Find all SSNs that appear in either the Employee table or the Works_On table.

✅ Show Solution

$$\pi_{Ssn}(Employee) \cup \pi_{Essn}(Works_On)$$

RELATIONAL ALGEBRA (RELAX)

```
(π Ssn (Employee) ∪ π Essn (Works_On))
```

SQL EQUIVALENT

```
SELECT DISTINCT Ssn FROM Employee
UNION
SELECT DISTINCT Essn FROM Works_On
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(π Ssn (Employee) ∪ π Essn (Works_On))
```

📝 EXAMPLE 12: — Union

Retrieve the Social Security Numbers (Ssn) for all employees who work in **Department 3** or directly supervise an employee in **Department 3**

✅ Show Solution

To solve this using **Relational Algebra**, we break the problem into two parts and combine them using the **Union** operator:

1. Find employees working in Department 3.
2. Find supervisors of employees working in Department 3
3. Combine results using Union.

$$R_1 = \pi_{\text{Ssn}}(\sigma_{\text{Dno}=3}(\text{Employee}))$$
$$R_2 = \pi_{\text{Super_ssn}}(\sigma_{\text{Dno}=3}(\text{Employee}))$$

$$\text{Result} = R_1 \cup R_2$$

RELATIONAL ALGEBRA (RELAX)

```
R_1 = π Ssn (σ Dno = 3 (Employee))
R_2 = π Super_ssn (σ Dno = 3 (Employee))
(R_1 ∪ R_2)
```

SQL EQUIVALENT

```
WITH R_1 AS (
  SELECT DISTINCT Ssn FROM (SELECT * FROM Employee WHERE Dno = 3) AS
t1
),
R_2 AS (
  SELECT DISTINCT Super_ssn FROM (SELECT * FROM Employee WHERE Dno =
3) AS t2
),
Result AS (
  SELECT * FROM R_1
UNION
SELECT * FROM R_2
)
SELECT * FROM Result
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
R_1 = π Ssn (σ Dno = 3 (Employee))
R_2 = π Super_ssn (σ Dno = 3 (Employee))
```

4.2 Intersection — ∩

📖 DEFINITION 3: **Intersection (R ∩ S)**

Returns only the tuples that are present in **both** relation R and relation S.

$$R \cap S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ∩ S)
```

SQL EQUIVALENT

```
SELECT * FROM R
INTERSECT
SELECT * FROM S
```

ⓘ **Note**

Intersection is a **derived** operation — it can always be rewritten using set difference.

$$R \cap S = R - (R - S)$$

📝 EXAMPLE 13: **Intersection**

Retrieve the Male Employees that work in department 1

✅ Show Solution

Steps:

- Retrieve all employees in department 1 as R1.

- Retrieve all Male employees in Employee as R2.
- Result = R1 INTERSECTION R2.

$$R_1 = \sigma_{\text{Dno}=1}(\text{Employee})$$
$$R_2 = \sigma_{\text{Gender}='M'}(\text{Employee})$$
$$\text{Result} = R_1 \cap R_2$$

RELATIONAL ALGEBRA (RELAX)

```
R_1 = σ Dno = 1 (Employee)
R_2 = σ Gender = 'M' (Employee)
(R_1 ∩ R_2)
```

SQL EQUIVALENT

```
WITH R_1 AS (
  SELECT * FROM Employee WHERE Dno = 1
),
R_2 AS (
  SELECT * FROM Employee WHERE Gender = 'M'
),
Result AS (
  SELECT * FROM R_1
INTERSECT
SELECT * FROM R_2
)
SELECT * FROM Result
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
R_1 = σ Dno = 1 (Employee)
R_2 = σ Gender = 'M' (Employee)
```

4.3 Set Difference — – (In One But Not the Other)

📖 DEFINITION 4: **Set Difference (R - S)**

Returns all tuples that are present in relation R but **not** present in relation S. Requires union compatibility.

$$R - S$$

RELATIONAL ALGEBRA (RELAX)

```
(R - S)
```

SQL EQUIVALENT

```
SELECT * FROM R
EXCEPT
SELECT * FROM S
```

📝 EXAMPLE 14: **Set Difference**

Find the SSNs of employees who are NOT currently assigned to any project.

✅ Show Solution

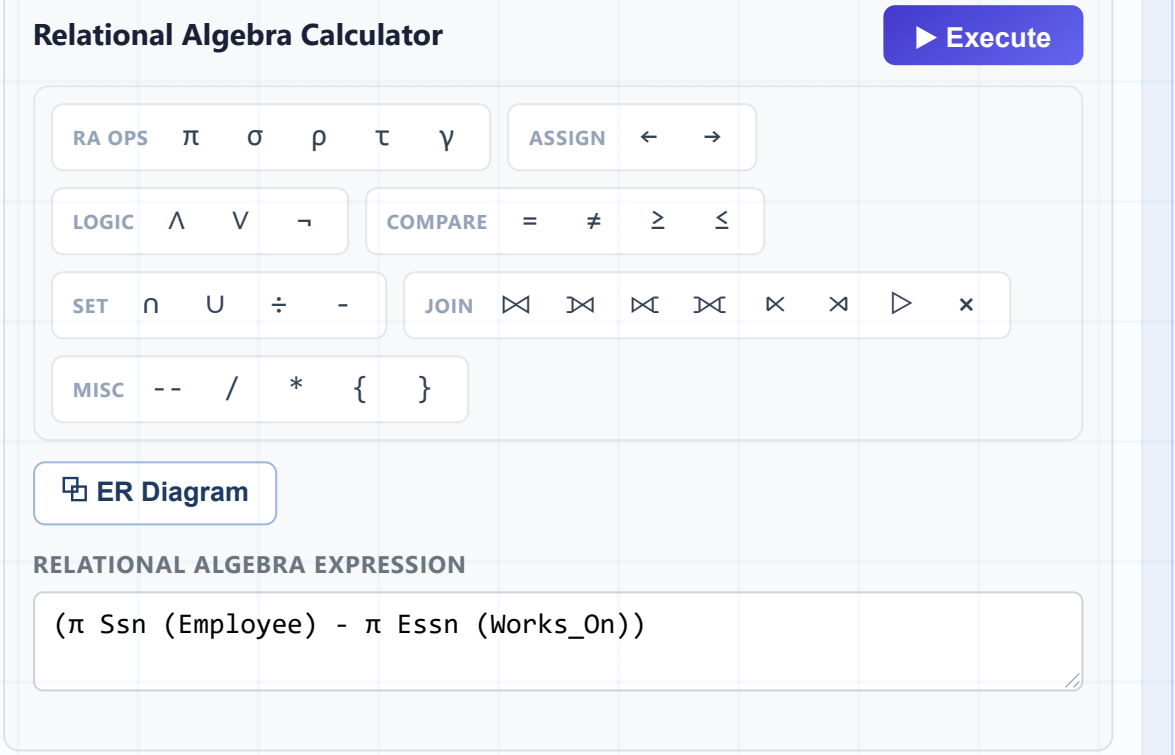
$$\pi_{Ssn}(Employee) - \pi_{Essn}(Works_On)$$

RELATIONAL ALGEBRA (RELAX)

```
(π Ssn (Employee) - π Essn (Works_On))
```

SQL EQUIVALENT

```
SELECT DISTINCT Ssn FROM Employee
EXCEPT
SELECT DISTINCT Essn FROM Works_On
```



5 Cartesian Product — × (All Combinations)

Unary relational algebra operations like **Select** and **Project** operate on a **single relation** at a time. However, in real-world relational databases, data is organized according to **normalization principles** to eliminate redundancy, avoid data anomalies, and maintain data integrity. As a result, related information is split across multiple tables (for example, storing employee details in an `Employee` relation and department details in a `Department` relation).

To answer queries that require information spanning multiple tables, we need a mechanism to correlate tuples across relations:

- **Combining Schemas:** The **Cartesian Product ($R \times S$)** serves as the fundamental relational algebra operation that pairs every tuple in R with every tuple in S. This generates a new relation containing the combined attributes of both relations (Degree: degree(R) + degree(S)).
- **The Foundation for Joins:** While a raw Cartesian Product generates all possible combinations—including many non-matching or irrelevant pairs—it forms the mathematical foundation for **Join operations**. When followed by a **Selection** on a matching key condition (such as R.Dno = S.Dnumber), it narrows down the combined dataset to extract meaningful, related records across normalized tables.

> **Summary**
>
> Without the Cartesian Product (and the join operations built upon it), a relational system could only query individual, isolated tables, defeating the core purpose of a normalized relational database design.

$$R \times S$$

RELATIONAL ALGEBRA (RELAX)

```
(R × S)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM R) AS t1 CROSS JOIN (SELECT * FROM S) AS t2
```

Table 23: *Relation R*

Table 24: *Relation S*

s1
7
8

Table 25: *R X S*

r1	r2
1	2
3	4
5	6

r1	r2	s1
1	2	7
1	2	8
3	4	7
3	4	8
5	6	7
5	6	8

Property	Value
Output degree	degree(R) + degree(S) — shared names qualified as R.col and S.col
Output cardinality	| R | × | S |

📝 EXAMPLE 15: Cartesian Product

Produce all combinations of Employee and Department rows. Notice the resulting degree (columns) and cardinality (rows).

✅ Show Solution

$$\text{Employee} \times \text{Department}$$

RELATIONAL ALGEBRA (RELAX)

```
(Employee × Department)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM Employee) AS t1 CROSS JOIN (SELECT *
FROM Department) AS t2
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Employee × Department)
```

> **Important**
>
> The raw Cartesian product generates **30 × 8 = 240 rows** but most are meaningless combinations. It is almost always followed by a σσ to keep only matching rows — that combination is precisely a **join**.

EXAMPLE 16: Column Name Conflict

Try to pair every employee with every other employee using a Cartesian product.

Show Solution

$$Employee \times Employee$$

RELATIONAL ALGEBRA (RELAX)

```
(Employee × Employee)
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Employee × Employee)
```

Why Employee × Employee Fails (Column Name Conflict)

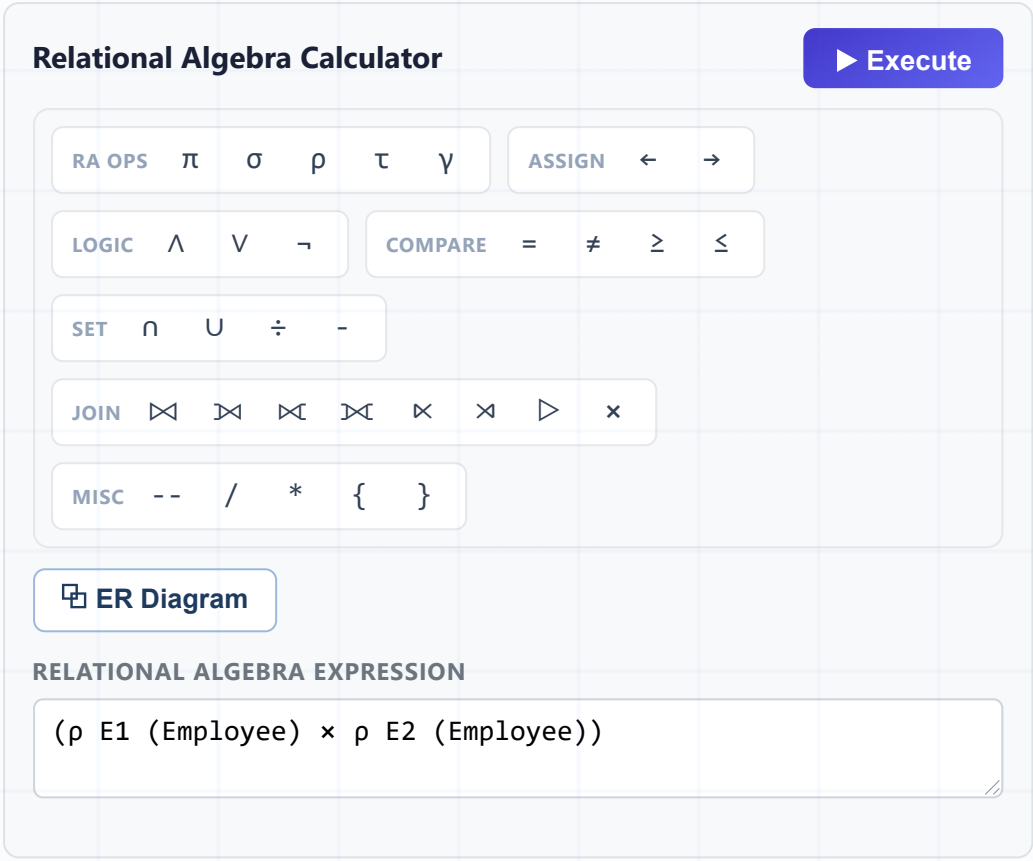
When you write Employee × Employee, both copies of the table bring the **same 10 column names** (`Fname` , `Lname` , `Ssn` , ...).
A relation must have **unique column names** — there is no way to tell `Fname` (left copy) from `Fname` (right copy).

The correct approach: use ρ to **rename** one (or both) copies before forming the product:

$$\rho_{E1}(Employee) \times \rho_{E2}(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
(ρ E1 (Employee) × ρ E2 (Employee))
```



The general rule

every column in a relation must be uniquely identifiable. Whenever a Cartesian product would produce duplicate column names, use ρ to rename one operand before the product.

5.1 Joining Multiple Tables using Foreign Keys

When information is spread across multiple tables, we must combine (merge) them to retrieve the data we need.

- **Why Unconditioned Merges Fail:** Performing a Cartesian product (merging tables without a condition) pairs every row of the first table with every row of the second

table, creating meaningless and illogical combinations.

- **Using Foreign Keys as Join Conditions:** To ensure we extract accurate and logical data, we must take advantage of foreign key relationships. Using the foreign key as a join condition (PK = FK) filters out invalid pairs, guaranteeing that only truly related records are merged.

📝 **EXAMPLE 17:**

Retrieve all Female dependent names for each employee.

✅ Show Solution

To retrieve the names of all dependents associated with female employees, you must combine the **Employee** relation and the **Dependent** relation using their foreign key relationship.

1. Perform the Cartesian Product:

$$\text{All Pairs} \leftarrow \text{Employee} \times \text{Dependent}$$

2. Apply the Selection (Filter for matching Foreign Key AND Female Gender):

$$\text{FemaleDependents} \leftarrow \sigma_{\text{Ssn}=\text{Essn} \wedge \text{Gender}=\text{'F'}}(\text{AllPairs})$$

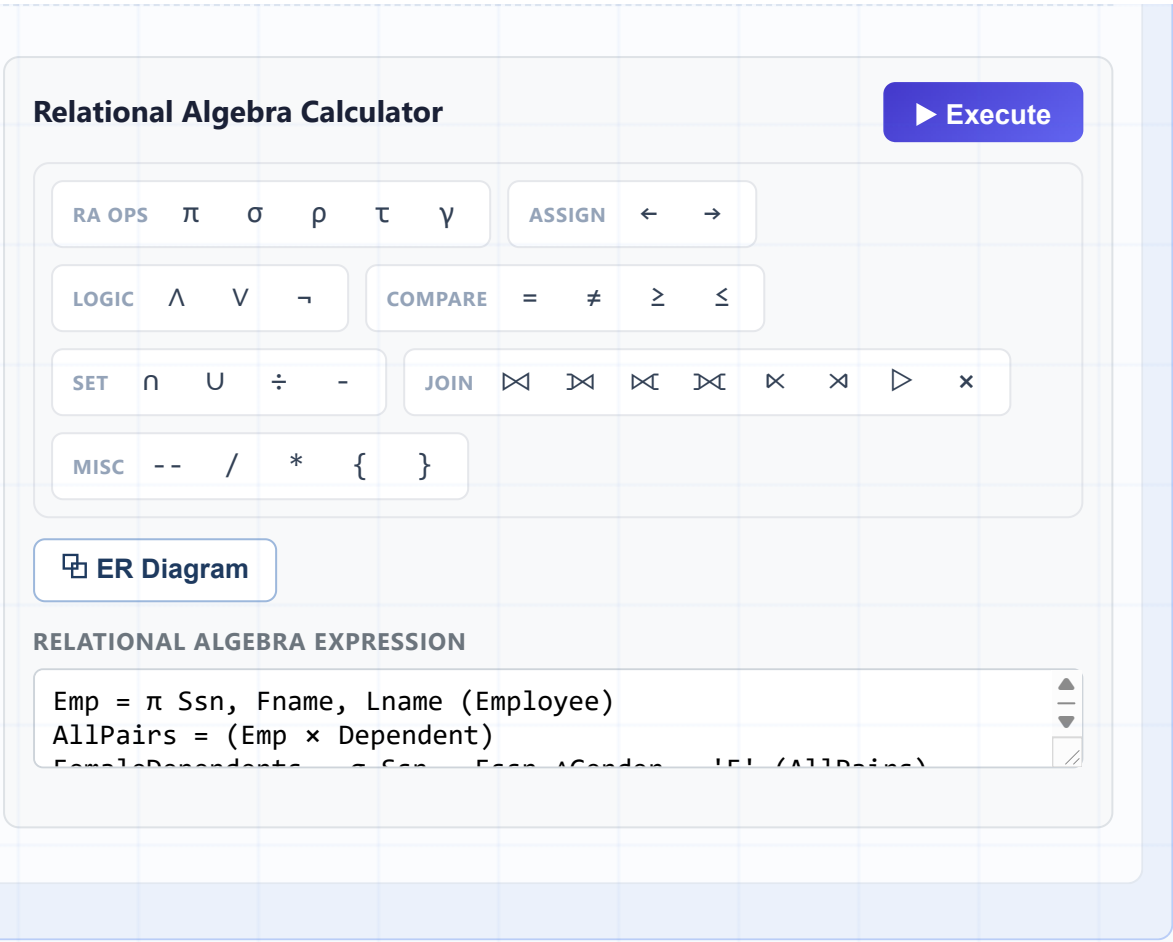
3. Project the Desired Attribute:

$$\text{Result} \leftarrow \pi_{\text{Dependent_name}}(\text{FemaleDependents})$$

$$Emp = \pi_{Ssn,Fname,Lname}(Employee)$$
$$\text{AllPairs} \leftarrow \text{Emp} \times \text{Dependent}$$
$$\text{FemaleDependents} \leftarrow \sigma_{\text{Ssn}=\text{Essn} \wedge \text{Gender}=\text{'F'}}(\text{AllPairs})$$
$$\text{Result} \leftarrow \pi_{\text{Dependent_name},Fname,Lname}(\text{FemaleDependents})$$

RELATIONAL ALGEBRA (RELAX)

```
Emp = π Ssn, Fname, Lname (Employee)
AllPairs = (Emp × Dependent)
FemaleDependents = σ Ssn = Essn ∧Gender = 'F' (AllPairs)
π Dependent_name, Fname, Lname (FemaleDependents)
```



6 Joins

📖 DEFINITION 5: **Join Operation**

A join operation in relational algebra combines tuples (rows) from two separate relations (tables) based on a specific condition, acting fundamentally as a Cartesian product followed by a selection.

6.1 Theta Join (Cartesian Product + Filter)

📖 DEFINITION 6: **A theta-join (⋈θ)**

A theta-join ($\bowtie_\theta$) is a relational algebra operation that combines two relations based on any general comparison or conditional operator, rather than strictly

equality. It is formally defined as performing a Cartesian product first and then immediately applying a selection condition (σ) to filter out unrelated combinations.

$$R \bowtie_{\theta} S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ⋈ [standard condition] S)
```

where:

- **General Comparison Operators:** Unlike an equi-join (which strictly uses equality `$=$`), a theta-join allows any standard condition operator ($\theta \in \{=, \neq, <, \leq, >, \geq\}$).
- **Formal Algebraic Equivalence:** Formally, a theta-join between two relations R and S is defined as performing a **Cartesian Product** first and then applying a **Selection** (σ) condition:

  $$R \bowtie_{\theta} S \equiv \sigma_{\theta}(R \times S)$$

  Where θ represents the join condition (e.g., $R.A > S.B$ or $R.Ssn = S.Essn$).

Tip

A **Theta Join** ($\bowtie_\theta$) is simply a general form of an **inner join**.

- **Mechanism:** It combines two relations by pairing rows that satisfy a specified condition θ (e.g., $=$, $\neq$, $<$, $>$, $\leq$, $\geq$).
- **Inner Join Behavior:** Only tuples that evaluate to **true** under condition θ are included in the result set; unmatched tuples are discarded.
- **Equivalence:** A Theta Join is formally defined as a Cartesian product followed by a Selection:

$$\text{Relation}_1 \bowtie_\theta \text{Relation}_2 \equiv \sigma_\theta(\text{Relation}_1 \times \text{Relation}_2)$$

- An **equijoin** is simply a specific type of **theta join** where the condition relies strictly on the equality operator ($=$). While all equijoins are theta joins, not all theta joins are equijoins (as theta joins can use any comparison operator, such as $>$, $<$, or $\neq$).

EXAMPLE 18:

List each employee with their project number and hours, joining Employee and Works_On on Ssn = Essn.

Show Solution

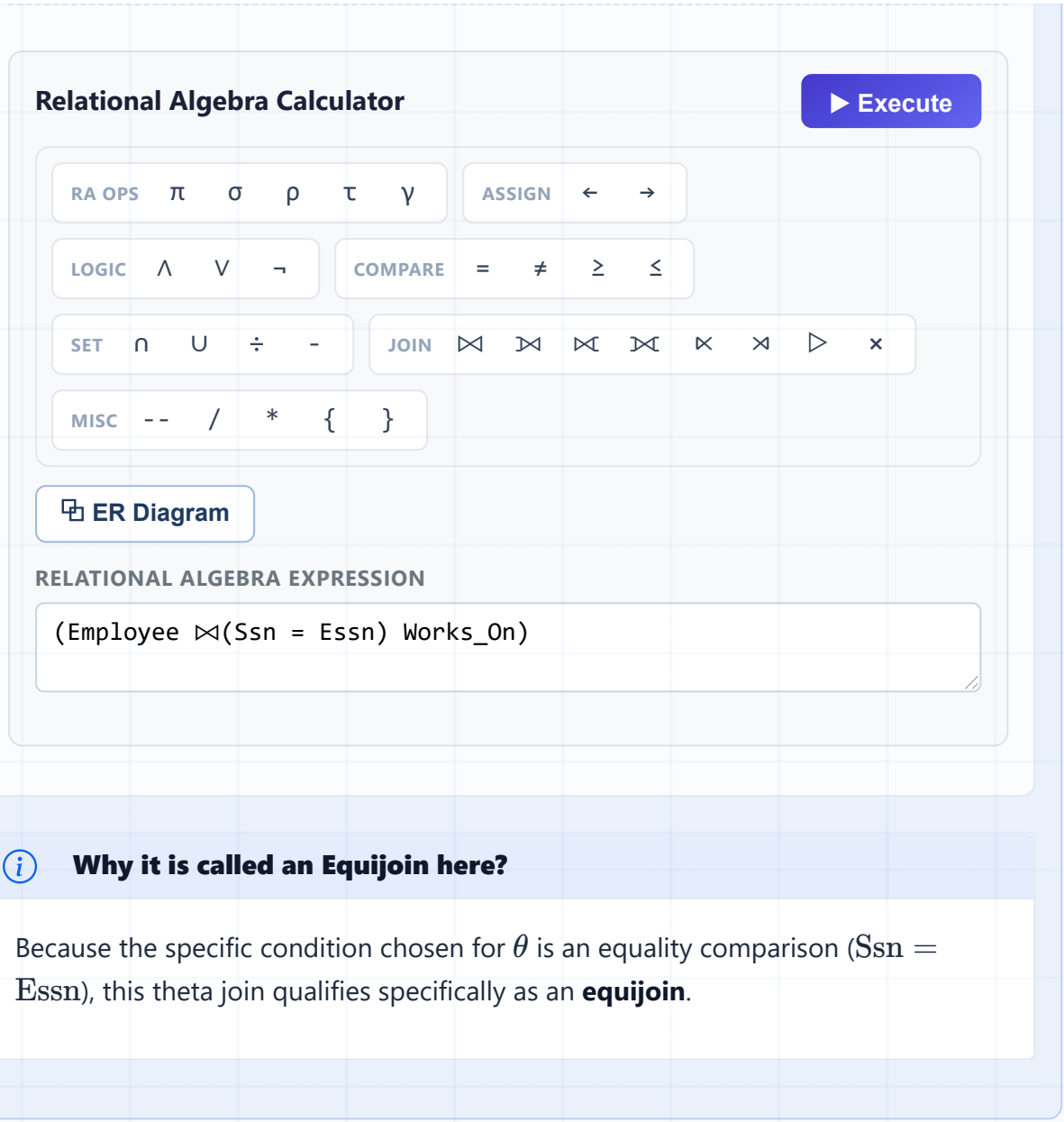
$$\text{Employee} \bowtie_{\text{Ssn}=\text{Essn}} \text{Works_On}$$

RELATIONAL ALGEBRA (RELAX)

```
(Employee ⋈(Ssn = Essn) Works_On)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM Employee) AS t1 JOIN (SELECT * FROM
Works_On) AS t2 ON Ssn = Essn
```



> **Why it is called an Equijoin here?**
>
> Because the specific condition chosen for θ is an equality comparison ($\mathrm{Ssn} = \mathrm{Essn}$), this theta join qualifies specifically as an **equijoin**.

6.2 Natural Join — ⋈

> 📖 DEFINITION 7: **Natural Join**
>
> a **Natural Join (⋈)** is a derived relational algebra operation that automatically performs a join based on all attributes that share the same name in both relations.

Key Properties:

- **Implicit Equality Condition:** It matches rows where all shared attribute names have equal values (an implicit equijoin).
- **Duplicate Column Elimination:** Unlike a Cartesian product or theta join, duplicate common attributes are automatically removed—appearing only once in the resulting relation's schema.

Result Schema Rule

If relation R has attributes (A, B, C) and relation S has attributes (B, C, D), then the natural join $R \bowtie S$ results in a relation with attributes (A, B, C, D).

$$R \bowtie S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ⋈ S)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM R) AS t1 NATURAL JOIN (SELECT * FROM S) AS t2
```

Natural Join is Name-Driven

Natural join joins on **every** column that shares the same name in both tables.

- If PK and FK have **different names** → no common columns → result is a **Cartesian product** (all combinations, probably wrong)
- If you accidentally share **too many** column names → the join condition becomes overly restrictive
- Always verify which columns will be matched before using natural join

EXAMPLE 19: **Natural Join: When It Works Directly**

List each department with all its office locations.

✅ Show Solution

- The `Department` table has `Dnumber` as its primary key.
- The `Dept_Locations` table has `Dnumber` as a foreign key.
- **Both columns share the exact same name** → natural join will automatically join on `Dnumber`.

$$Department \bowtie Dept_Locations$$

RELATIONAL ALGEBRA (RELAX)

```
(Department ⋈ Dept_Locations)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM Department) AS t1 NATURAL JOIN (SELECT * FROM Dept_Locations) AS t2
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Department ⋈ Dept_Locations)
```

📝 EXAMPLE 20: **Natural Join: When Column Names Differ (Danger!)**

Try to list each employee with the name of their department using natural join.

✅ Show Solution

`Employee.Dno` is the FK referencing `Department.Dnumber` — but the column names are **different**.

Since there are **no columns with identical names** between Employee and Department, a natural join produces a **Cartesian product** — every employee paired with every department, regardless of which department they belong to.

$$Employee \bowtie Department$$

RELATIONAL ALGEBRA (RELAX)

```
(Employee ⋈ Department)
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Employee ⋈ Department)
```

Fix with Rename ρ: Force Natural Join

Strategy: Use ρ to rename `Dnumber` to `Dno` in Department, making the column name match.

$$Deps = \rho_{Dno \leftarrow Dnumber} Department$$
$$Employee \bowtie Deps$$

RELATIONAL ALGEBRA (RELAX)

```
Deps = ρ Dno←Dnumber (Department)
(Employee ⋈ Deps)
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ -

JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
Deps = ρ Dno←Dnumber (Department)
(Employee ⋈ Deps)
```

6.3 Outer Joins

Standard (inner) joins **discard unmatched rows**. Outer joins **preserve** them by filling missing values with NULL.

Operator	Symbol	Keeps unmatched from
Left outer join	R⟕S	R (left operand)
Right outer join	R⟖S	S (right operand)
Full outer join	R ⟗ S	Both R R and S S

6.3.1 Left Outer Join(R ⟕ S)

> 📖 **DEFINITION 8: Left Outer Join**
>
> A **Left Outer Join** is an extended relational algebra operation that returns all matching tuples from both relations, plus all unmatched tuples from the left relation.

$$R ⟕ S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ⟕ S)
```

Key Properties:

- **Preserves Left Relation Tuples:** Guarantees that every tuple from the left relation appears in the result at least once.
- `NULL` **Padding for Unmatched Rows:** If a tuple in R has no matching tuple in S based on the join condition, it is included in the output schema with `NULL` values for all attributes originating from S.

- **Schema / Output Columns:** The output schema contains all attributes from both R and S.

EXAMPLE 21:

Retrieve all employees along with their corresponding department information, even if some employees do not belong to any department.

Show Solution

$$Employee \bowtie_{Dno=Dnumber} Department$$

RELATIONAL ALGEBRA (RELAX)

```
(Employee ⋈(Dno = Dnumber) Department)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM Employee) AS t1 LEFT JOIN (SELECT * FROM Department) AS t2 ON Dno = Dnumber
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Employee ⋈(Dno = Dnumber) Department)
```

6.3.2 Right Outer Join($R \ltimes S$)

📖 **DEFINITION 9:** **Right Outer Join**

A **Right Outer Join** is an extended relational algebra operation that returns all matching tuples from both relations, plus all unmatched tuples from the right relation.

$$R \ltimes S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ⋉ S)
```

Key Properties:

- **Preserves Right Relation Tuples:** Guarantees that every tuple from the right relation appears in the result at least once.
- `NULL` **Padding for Unmatched Rows:** If a tuple in S has no matching tuple in R based on the join condition, it is included in the output schema with `NULL` values for all attributes originating from R.
- **Schema / Output Columns:** The output schema contains all attributes from both R and S.

📝 **EXAMPLE 22:**

Retrieve all departments along with their corresponding employee information, even if some departments do not have any employees assigned to them.

✅ Show Solution

$$Department \bowtie_{Dno=Dnumber} Employee$$

RELATIONAL ALGEBRA (RELAX)

```
(Department ⋈(Dno = Dnumber) Employee)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM Department) AS t1 RIGHT JOIN (SELECT *
FROM Employee) AS t2 ON Dno = Dnumber
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
(Department ⋈(Dno = Dnumber) Employee)
```

6.3.3 Full Outer Join(R ⟗ S)

📖 DEFINITION 10: **Full Outer Join**

A Full Outer Join is an extended relational algebra operation that combines the results of both a **Left Outer Join** and a **Right Outer Join**. It returns all matching tuples from both relations, along with all unmatched tuples from both the left and right relations.

$$R \rtimes S$$

RELATIONAL ALGEBRA (RELAX)

```
(R ⋉ S)
```

Key Properties:

- **Preserves All Tuples:** Guarantees that every tuple from both relations (R and S) appears in the result at least once
- Unmatched tuples from the left relation (R) are padded with `NULL` values for all attributes of the right relation (S).
- Unmatched tuples from the right relation (S) are padded with `NULL` values for all attributes of the left relation (R)
- **Schema / Output Columns:** The output schema contains all attributes from both relations R and S.

📝 EXAMPLE 23:

List all employees, all project assignments, and all projects—including employees who are not assigned to any project, project assignment records without a matching employee, and projects that have zero employees assigned to them.

✅ Show Solution

$$\text{Employee} \rtimes_{\text{Ssn}=\text{Essn}} \text{Works_On} \rtimes_{\text{Pno}=\text{Pnumber}} \text{Project}$$

RELATIONAL ALGEBRA (RELAX)

```
((Employee ⋈(Ssn = Essn) Works_On) ⋈(Pno = Pnumber) Project)
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT * FROM (SELECT * FROM Employee) AS t1 FULL
OUTER JOIN (SELECT * FROM Works_On) AS t2 ON Ssn = Essn) AS t3 FULL
OUTER JOIN (SELECT * FROM Project) AS t4 ON Pno = Pnumber
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
((Employee ⋈(Ssn = Essn) Works_On) ⋈(Pno = Pnumber) Project)
```

7 Extended Relational Algebra — Aggregation (γ)

Standard relational algebra operates on **sets** of tuples with no grouping or counting. Real queries often need summaries: “count employees per department”, “average salary by gender”, “maximum project hours”. The **extended relational algebra** adds the **aggregate operator** γ (gamma).

$$\gamma_G; F(A) \rightarrow N\ (R)$$

RELATIONAL ALGEBRA (RELAX)

where:

- γ **(Gamma):** Represents the extended relational algebra aggregation operator.
- **G is grouping attribute** (equivalent to SQL `GROUP BY`).
- **F(A):** The aggregate function **F** applied to attribute **A**.
- **N**: alias/name to the computed result.
- R**:** The input relation/table or subquery.

Table 26: Aggregate functions

Function	Meaning
COUNT()	Count rows in each group
SUM(col)	Sum of values
AVG(col)	Average of values
MIN(col)	Minimum value
MAX(col)	Maximum value

📝 EXAMPLE 24: **Count Employees per Department**

How many employees work in each department?

✅ Show Solution

$$\gamma Dno; COUNT() \rightarrow NumEmp(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
γ Dno;COUNT(*)→NumEmp (Employee)
```

SQL EQUIVALENT

```
SELECT Dno, COUNT(*) AS NumEmp FROM Employee GROUP BY Dno
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
γ Dno;COUNT(*)→NumEmp (Employee)
```

EXAMPLE 25: Average Salary per Department

What is the average salary in each department?

Show Solution

$$\gamma Dno; AVG(Salary) \rightarrow AvgSalary(Employee)$$

RELATIONAL ALGEBRA (RELAX)

```
γ Dno;AVG(Salary)→AvgSalary (Employee)
```

SQL EQUIVALENT

```
SELECT Dno, AVG(Salary) AS AvgSalary FROM Employee GROUP BY Dno
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
γ Dno;AVG(Salary)→AvgSalary (Employee)
```

EXAMPLE 26: Multiple Aggregates: Max and Min Salary per Department

In each department, what is the highest and lowest salary?

Show Solution

$$\gamma Dno; MAX(Salary) \rightarrow MaxSal, MIN(Salary) \rightarrow MinSal(Employ$$

RELATIONAL ALGEBRA (RELAX)

```
γ Dno;MAX(Salary)→MaxSal,MIN(Salary)→MinSal (Employee)
```

SQL EQUIVALENT

```
SELECT Dno, MAX(Salary) AS MaxSal, MIN(Salary) AS MinSal FROM
Employee GROUP BY Dno
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
γ Dno;MAX(Salary)→MaxSal,MIN(Salary)→MinSal (Employee)
```

EXAMPLE 27:

Aggregation with Filter: Departments Averaging Over $33,000

List department numbers where the average salary exceeds $33,000.

✅ Show Solution

$$\sigma_{AvgSalary>33000}(\gamma Dno; AVG(Salary) \rightarrow AvgSalary(Employee))$$

RELATIONAL ALGEBRA (RELAX)

```
σ AvgSalary > 33000 (γ Dno;AVG(Salary)→AvgSalary (Employee))
```

SQL EQUIVALENT

```
SELECT * FROM (SELECT Dno, AVG(Salary) AS AvgSalary FROM Employee
GROUP BY Dno) AS t1 WHERE AvgSalary > 33000
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ AvgSalary > 33000 (γ Dno;AVG(Salary)→AvgSalary (Employee))
```

EXAMPLE 28: Total Hours per Employee Across All Projects

For each employee, calculate the total hours worked across all their project assignments.

Show Solution

$$\gamma Essn; SUM(Hours) \rightarrow TotalHours(Works_On)$$

RELATIONAL ALGEBRA (RELAX)

```
γ Essn;SUM(Hours)→TotalHours (Works_On)
```

SQL EQUIVALENT

```
SELECT Essn, SUM(Hours) AS TotalHours FROM Works_On GROUP BY Essn
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
γ Essn;SUM(Hours)→TotalHours (Works_On)
```

8 Exercises

8.1 University Database

8.1.1 University Database Schema Overview

The **University Database** schema is designed to model core academic and administrative operations within a higher education institution. It manages information across six key entities: students, faculty, departments, courses, course enrollments, and teaching assignments.

8.1.1.1 Entity Descriptions & Key Relationships

- `Department`
  - **Attributes:** `Did` (Primary Key), `Dname`, `Chair_id` (Foreign Key → `Faculty.Fid`), `Building`.
  - **Description:** Represents academic departments within the university. Each department operates in a specific building and is chaired by a faculty member.
- `Faculty`
  - **Attributes:** `Fid` (Primary Key), `Fname`, `Lname`, `Rank`, `Salary`, `Dept_id` (Foreign Key → `Department.Did`).
  - **Description:** Represents academic staff members. Each faculty member belongs to a specific department and holds an academic rank (e.g., Professor, AssocProf, AssistProf).
- `Student`
  - **Attributes:** `Sid` (Primary Key), `Fname`, `Lname`, `Email`, `Major`, `GPA`, `Year`, `Advisor_id` (Foreign Key → `Faculty.Fid`).
  - **Description:** Stores student demographic and academic performance data. Each student is assigned a faculty member as their academic advisor.
- `Course`
  - **Attributes:** `Cid` (Primary Key), `Cname`, `Credits`, `Dept_id` (Foreign Key → `Department.Did`).
  - **Description:** Defines subjects offered by the university. Every course belongs to an offering department and carries a specified number of academic credit units.

- `Enrollment` *(Junction Table for Student–Course $M:N$ Relationship)*
  - **Attributes:** `Sid` (Foreign Key $\rightarrow$ `Student.Sid`), `Cid` (Foreign Key $\rightarrow$ `Course.Cid`), `Semester`, `Grade`.
  - **Composite Primary Key:** (`Sid`, `Cid`, `Semester`)
  - **Description:** Tracks student course registrations and academic performance (letter grades) across different academic terms.
- `Teaches` *(Junction Table for Faculty–Course $M:N$ Relationship)*
  - **Attributes:** `Fid` (Foreign Key $\rightarrow$ `Faculty.Fid`), `Cid` (Foreign Key $\rightarrow$ `Course.Cid`), `Semester`, `Room`.
  - **Composite Primary Key:** (`Fid`, `Cid`, `Semester`)
  - **Description:** Maps teaching responsibilities, indicating which faculty member teaches a specific course section, semester, and assigned classroom location.

8.1.1.2 Schema Reference

- **Student** (Sid, Fname, Lname, Email, Major, GPA, Year, Advisor_id)
- **Faculty** (Fid, Fname, Lname, Rank, Salary, Dept_id)
- **Department** (Did, Dname, Chair_id, Building)
- **Course** (Cid, Cname, Credits, Dept_id)
- **Enrollment** (Sid, Cid, Semester, Grade)
- **Teaches** (Fid, Cid, Semester, Room)

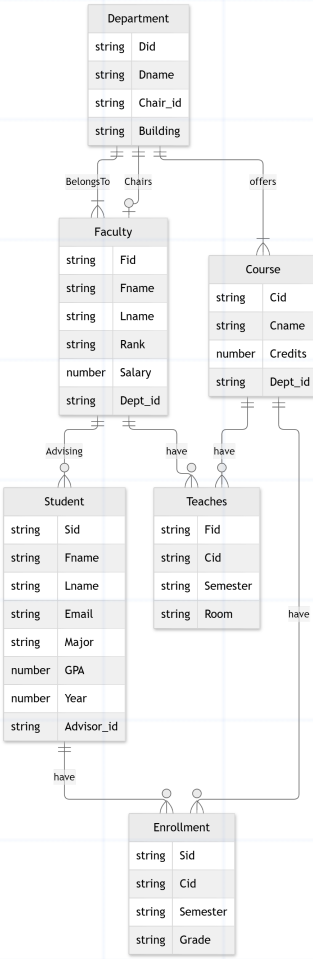


Diagram 57: University Database

8.1.2 Exercises

🖍️ EXERCISE 1: Selection & Projection (Basic)

Find the first name, last name, and GPA of all **Computer Science ('CS')** students who have a GPA higher than **3.5**.

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬

COMPARE = ≠ ≥ ≤ | SET ∩ ∪ ÷ -

JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Salary > 60000 (Employee)
```

✅ Show Solution

$$\pi_{\text{Fname,Lname,GPA}}\left(\sigma_{\text{Major='CS'} \wedge \text{GPA}>3.5}(\text{Student})\right)$$

RELATIONAL ALGEBRA (RELAX)

```
π Fname, Lname, GPA (σ Major = 'CS' ∧GPA > 3.5 (Student))
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π Fname, Lname, GPA (σ Major = 'CS' ∧GPA > 3.5 (Student))
```

🖍️ EXERCISE 2: **Theta Join / Equijoin**

Find the full names ($\text{Fname}, \text{Lname}$) of all students along with the last name (Lname) of their academic advisor.

Relational Algebra Calculator

▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬

COMPARE = ≠ ≥ ≤ | SET ∩ ∪ ÷ -

JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Salary > 60000 (Employee)
```

✅ Show Solution

$$\pi_{\text{Student.Fname,Student.Lname,Faculty.Lname}} \text{ Student } \bowtie_{\text{Advisor_id=Fid}} \text{ Faculty}$$

RELATIONAL ALGEBRA (RELAX)

```
π Student.Fname, Student.Lname, Faculty.Lname ((Student ⋈(Advisor_id
= Fid) Faculty))
```


Relational Algebra Calculator

▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬ | COMPARE = ≠ ≥ ≤ | SET ∩ ∪ ÷ - | JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π Student.Fname, Student.Lname, Faculty.Lname ((Student
⋈(Advisor_id = Fid) Faculty))
```

🖍️ EXERCISE 3: Natural Join & Multi-Table Join

List the names of all courses (Cname) offered by the 'Computer Science' department, including the department name (Dname).

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ | ASSIGN ← → | LOGIC ∧ ∨ ¬

COMPARE = ≠ ≥ ≤ | SET ∩ ∪ ÷ -

JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × | MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Salary > 60000 (Employee)
```

✅ Show Solution

$$\pi_{\text{Cname},\text{Dname}} (\text{Course} \bowtie \sigma_{\text{Dname}='\text{Computer Science}'}(\text{Department}))$$

RELATIONAL ALGEBRA (RELAX)

```
π Cname, Dname ((Course ⋈ σ Dname = 'Computer Science'
(Department)))
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π Cname, Dname ((Course ⋈ σ Dname = 'Computer Science' (Department)))
```

EXERCISE 4: Outer Joins

List all faculty members ($\mathrm{Fname}, \mathrm{Lname}$) and the course IDs ($\mathrm{Cid}$) they teach, including faculty members who are not currently teaching any courses.

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬

COMPARE = ≠ ≥ ≤ SET ∩ ∪ ÷ -

JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Salary > 60000 (Employee)
```

Show Solution

$$\pi_{\text{Fname,Lname,Cid}}\left(\text{Faculty} \bowtie_{\text{Faculty.Fid}=\text{Teaches.Fid}} \text{Teaches}\right)$$

RELATIONAL ALGEBRA (RELAX)

```
π Fname, Lname, Cid ((Faculty ⋈(Faculty.Fid = Teaches.Fid) Teaches))
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
π Fname, Lname, Cid ((Faculty ⋈(Faculty.Fid = Teaches.Fid)
Teaches))
```

EXERCISE 5: Aggregation

Find the average GPA and the total number of students for each Major.

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← → LOGIC ∧ ∨ ¬

COMPARE = ≠ ≥ ≤ SET ∩ ∪ ÷ -

JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ × MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
σ Salary > 60000 (Employee)
```

Show Solution

$$\gamma_{\text{Major};\text{AVG(GPA)}\rightarrow\text{AvgGPA},\ \text{COUNT(Sid)}\rightarrow\text{TotalStudents}}(\text{Student})$$

RELATIONAL ALGEBRA (RELAX)

```
γ Major;AVG(GPA)→AvgGPA, COUNT(Sid)→TotalStudents (Student)
```

Relational Algebra Calculator ▶ Execute

RA OPS π σ ρ τ γ ASSIGN ← →

LOGIC ∧ ∨ ¬ COMPARE = ≠ ≥ ≤

SET ∩ ∪ ÷ - JOIN ⋈ ⋈ ⋈ ⋈ ⋉ ⋊ ▷ ×

MISC -- / * { }

ER Diagram

RELATIONAL ALGEBRA EXPRESSION

```
γ Major;AVG(GPA)→AvgGPA, COUNT(Sid)→TotalStudents (Student)
```